

CS 1101 Unit 2

Programming Assignment

Assignment Response

1. Circumference Calculation

The circumference of a circle is calculated by $2\pi r$, where $\pi = 3.14159$ (rounded to five decimal places). Write a function called `print_circum` that takes an argument for the circle's radius and prints the circle's circumference. Call your `print_circum` function three times with different values for radius.

Include the following in your part 1 submission:

- * The code for your `print_circum` function.
- * A screenshot showing inputs and outputs to three calls of your `print_circum`.
- * The code and its output must be explained technically.
- * The explanation can be provided before or after the code, or in the form of comments within the code.

```
# import math

def print_circum(radius):

    pi = 3.14159 # Value specified in the assignment

    # or pi = math.pi

    circumference = 2 * pi * radius

    print(f"Radius: {radius} -> Circumference: {circumference}")
```

```
# Calling the function three times with different values
```

```
print_circum(5)
```

```
print_circum(10.5)
```

```
print_circum(21)
```

The screenshot shows a VS Code editor with a Python file named `circumferenceCalculation.py`. The code defines a function `print_circum(radius)` that calculates the circumference of a circle using the formula $C = 2 \times \pi \times r$. The function is called three times with different radius values: 5, 10.5, and 21. The terminal output shows the results of these calculations.

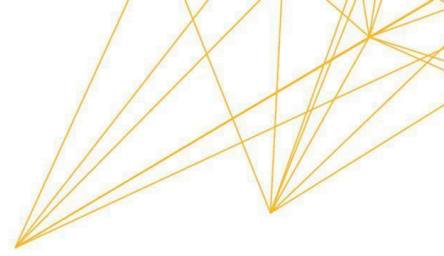
```
1 # import math
2
3 # Part 1: Circumference Calculation
4
5 def print_circum(radius):
6     """
7     Calculates and prints the circumference of a circle.
8     Formula: 2 * pi * r
9     """
10    pi = 3.14159 # Value specified in the assignment
11    # or pi = math.pi
12    circumference = 2 * pi * radius
13    print(f"Radius: {radius} -> Circumference: {circumference}")
14
15 # Calling the function three times with different values
16 print_circum(5)
17 print_circum(10.5)
18 print_circum(21)
```

The terminal output shows the results of the function calls:

```
Radius: 5 -> Circumference: 31.4159
Radius: 10.5 -> Circumference: 65.97339
Radius: 21 -> Circumference: 131.94678
```

```
~/Doc/UoPeopleCS1101-01ProgrammingFun
Radius: 5 -> Circumference: 31.4159
Radius: 10.5 -> Circumference: 65.97339
Radius: 21 -> Circumference: 131.94678
```

This program demonstrates the use of functions to perform specific tasks. As described by Downey (2015), “*A function definition specifies the name of a new function and the sequence of statements that run when the function is called*”. The first line of my code, `def`



`print_circum(radius):`, is called the header, and the indented lines that follow are called the body. In this function, `radius` is a parameter. According to Downey (2015), *“Inside the function, the arguments are assigned to variables called parameters”*. When I call the function using `print_circum(5)`, the value 5 is the argument passed to the function. I have also included notes starting with the `#` symbol. These are called comments, and they explain in natural language what the program is doing. For instance, I included a comment `# or pi = math.pi` to show that while I am using the hardcoded value 3.14159 as requested, I am aware that the `math` module contains a variable `math.pi` that stores a more precise value of `pi`. Because this function prints the result directly rather than returning it to the caller, it is considered a "void function".

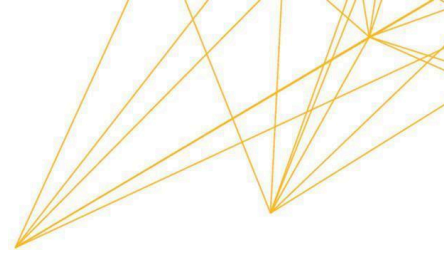
2. Catalog Project.

Develop a catalog for a company. Assume that this company sells three different Items. The seller can sell individual items or a combination of any two items. A gift pack is a special combination that contains all three items.

Here are some special considerations:

- A. If a customer purchases individual items, he does not receive any discount.
- B. If a customer purchases a combo pack with two unique items, he gets a 10% discount.
- C. If the customer purchases a gift pack, he gets a 25% discount.

```
def generate_catalog():  
    # Define base prices for the three items  
    item1 = 200.0  
    item2 = 400.0
```



```
item3 = 600.0

# Calculate discounted combos (10% discount)
# Formula: (Price A + Price B) * 0.90
combo1 = (item1 + item2) * 0.9
combo2 = (item2 + item3) * 0.9
combo3 = (item1 + item3) * 0.9

# Calculate Gift Pack (Combo 4) with 25% discount
# Formula: (Price A + Price B + Price C) * 0.75
gift_pack = (item1 + item2 + item3) * 0.75

# Display the catalog in the requested format
print("Online Store")
print("-" * 30)
print(f"{'Product(S)':<25} {'Price'}")
print(f"{'Item 1':<25} {item1}")
print(f"{'Item 2':<25} {item2}")
print(f"{'Item 3':<25} {item3}")
print(f"{'Combo 1(Item 1 + 2)':<25} {combo1}")
print(f"{'Combo 2(Item 2 + 3)':<25} {combo2}")
print(f"{'Combo 3(Item 1 + 3)':<25} {combo3}")
print(f"{'Combo 4(Item 1 + 2 + 3)':<25} {gift_pack}")
print("-" * 30)
print("For delivery Contact:98764678899")

# Execute the function to show output
generate_catalog()
```



The screenshot shows a VS Code editor window with a Python file named `generate_catalog.py`. The code defines a function `generate_catalog()` that calculates prices for three items and their combinations with discounts. The terminal output shows the results of running the script.

```
def generate_catalog():
    # Define base prices for the three items
    item1 = 200.0
    item2 = 400.0
    item3 = 600.0

    # Calculate discounted combos (10% discount)
    # Formula: (Price A + Price B) * 0.90
    combo1 = (item1 + item2) * 0.9
    combo2 = (item2 + item3) * 0.9
    combo3 = (item1 + item3) * 0.9

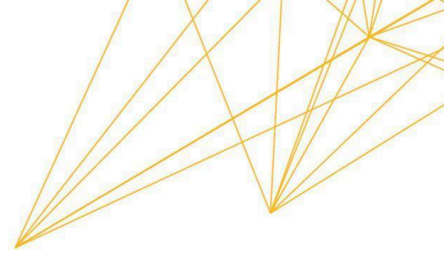
    # Calculate Gift Pack (Combo 4) with 25% discount
    # Formula: (Price A + Price B + Price C) * 0.75
    gift_pack = (item1 + item2 + item3) * 0.75

    # Display the catalog in the requested format
    print("Online Store")
    print("-" * 30)
    print(f"{'Product(S)':<25} {'Price'}")
    print(f"{'Item 1':<25} {item1}")
    print(f"{'Item 2':<25} {item2}")
    print(f"{'Item 3':<25} {item3}")
    print(f"{'Combo 1(Item 1 + 2)':<25} {combo1}")
    print(f"{'Combo 2(Item 2 + 3)':<25} {combo2}")
    print(f"{'Combo 3(Item 1 + 3)':<25} {combo3}")
    print(f"{'Combo 4(Item 1 + 2 + 3)':<25} {gift_pack}")
    print("-" * 30)
    print("For delivery Contact:98764678899")
```

```
Online Store
-----
Product(S)                Price
Item 1                    200.0
Item 2                    400.0
Item 3                    600.0
Combo 1(Item 1 + 2)       540.0
Combo 2(Item 2 + 3)       900.0
Combo 3(Item 1 + 3)       720.0
Combo 4(Item 1 + 2 + 3)   900.0
-----
For delivery Contact:98764678899
```

```
~/Doc/UoPeopleCS1101-01Program
Online Store
-----
Product(S)                Price
Item 1                    200.0
Item 2                    400.0
Item 3                    600.0
Combo 1(Item 1 + 2)       540.0
Combo 2(Item 2 + 3)       900.0
Combo 3(Item 1 + 3)       720.0
Combo 4(Item 1 + 2 + 3)   900.0
-----
For delivery Contact:98764678899
```

The `generate_catalog` function illustrates several key programming concepts. First, I created variables such as `item1`, `item2`, and `gift_pack` inside the function. As Downey (2015) notes, “When you create a variable inside a function, it is local, which means it only exists inside that function”. These variables store the floating-point values for the prices. To calculate the discounts, I wrote expressions using operators. “An expression is a combination of values,



variables, and operators” (Downey, 2015). For the combination packs, I had to be careful with the order of operations. *“Python follows mathematical conventions, often remembered by the acronym PEMDAS”* (Downey, 2015). For example, in the line `(item1 + item2) * 0.9`, *“Parentheses are used because they have the highest precedence”* (Downey, 2015). This forces the addition of the two items to happen before the multiplication by 0.9; otherwise, only item2 would have been multiplied. Finally, the catalog is printed using string formatting. I also used the multiplication operator on strings, such as `'-' * 30`, *“which performs repetition”* (Downey, 2015) to create the dividing lines.

References

Downey, A. (2015). Think Python: How to think like a computer scientist. Green Tree Press.
<https://greenteapress.com/thinkpython2/thinkpython2.pdf>